

FULL STACK DEVELOPMENT WITH MERN

BOOK A DOCTOR

Project Documentation

Project Title	Book A Doctor
Project Type	Full Stack Web Application
Technology	MERN Stack
Submitted By	1. Satya Navya Varalakshmi Mallidi, 2. Uppuluri Veera Shanmukha Krishna Koushik, 3. Akula Asha Rohini, 4. Surada Chinna, 5. Naramamidi Jyothi
Department	Electronics and Communication Engineering
Academic Year	2025–2026

1. Introduction

Project Title: Book A Doctor

Team Members: [Satya Navya Varalakshmi Mallidi, Uppuluri Veera Shanmukha Krishna Koushik, Akula Asha Rohini, Surada Chinna, Naramamidi Jyothi.]

Book A Doctor is a full stack healthcare appointment management web application. The system connects patients, doctors, and administrators through dedicated interfaces. It simplifies doctor onboarding, patient registration, appointment booking, and doctor-side appointment management.

2. Project Overview

Purpose

The purpose of the project is to provide a simple digital platform where patients can find available doctors and request appointments. Administrators can create and manage doctor accounts, while doctors can review appointment requests and patient information.

Key Features

- Separate login and role-based interfaces for administrators, doctors, and patients.
- Admin control center for adding doctor accounts and medical practice details.
- Patient account registration and authentication.
- Doctor listing with specialization and availability information.
- Online appointment booking with reason for consultation and additional notes.
- Doctor dashboard for appointment requests, appointment history, availability, and patient management.
- Patient management interface for viewing patient details.

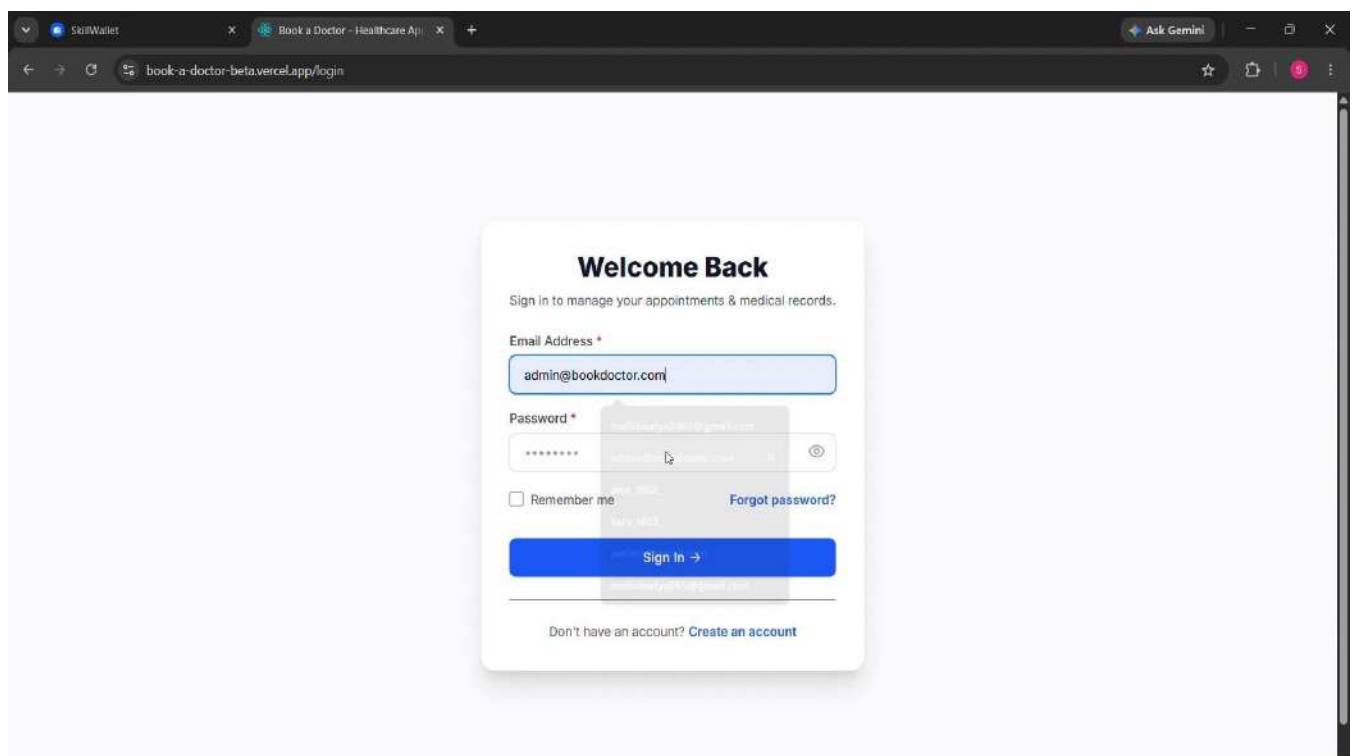


Figure 1: Admin login screen

3. Architecture

Frontend – React

The frontend is designed as a component-based React application. Reusable pages and UI components are used for authentication, dashboards, forms, doctor cards, appointment workflows, navigation, and role-specific views. The client communicates with backend APIs to retrieve and update application data.

Backend – Node.js and Express.js

The backend follows a REST API approach using Node.js and Express.js. Routes receive client requests, controllers process application logic, middleware handles authentication and authorization, and models interact with the database.

Database – MongoDB

MongoDB stores application data such as users, doctors, patients, appointments, specialization details, availability, and account information. Mongoose-style models can be used to define schemas and manage database interactions.

Layer	Responsibility
React Client	UI, forms, dashboards, booking workflow
Express API	Routes, request handling, business logic
Authentication Middleware	Identity verification and role access
MongoDB	Persistent users, doctors, patients, appointments

4. Setup Instructions

Prerequisites

- Node.js and npm
- MongoDB or MongoDB Atlas
- Git
- A modern web browser
- Code editor such as Visual Studio Code

Installation

- Clone the project repository using Git.
- Open the project folder in the code editor.
- Navigate to the client directory and run **npm install**.
- Navigate to the server directory and run **npm install**.
- Create the required environment file and configure the database connection, token secret, and application settings.
- Start the backend server and then start the React frontend.

5. Folder Structure

```
book-a-doctor/  
  client/  
    src/  
      components/  
      pages/  
      services/  
      assets/  
      App.js  
  server/  
    controllers/  
    models/  
    routes/  
    middleware/  
    config/  
    server.js
```

Client: Contains the React user interface, reusable components, pages, assets, and API service logic. **Server:** Contains backend routes, controllers, models, middleware, configuration, and server startup logic.

6. Running the Application

- Frontend: open the client directory and run **npm start**.
- Backend: open the server directory and run **npm start**.
- Open the local frontend URL displayed in the terminal and use the appropriate role login or registration page.

7. API Documentation

Method	Endpoint	Purpose
POST	/api/auth/login	Authenticate a user
POST	/api/patients/register	Create patient account
POST	/api/doctors	Create doctor account
GET	/api/doctors	Retrieve doctors
GET	/api/doctors/:id	Retrieve doctor details
POST	/api/appointments	Create appointment request
GET	/api/appointments	Retrieve appointments
PATCH	/api/appointments/:id	Update appointment status
GET	/api/patients	Retrieve patient information

Note: Endpoint names above are documented according to the demonstrated application workflow. They should be updated to match the exact route names in the final source code if the implementation uses different paths.

8. Authentication

The application uses authenticated access and role-based authorization. After login, the server verifies the supplied credentials and the application provides access to the correct dashboard based on the user's role.

Protected backend routes should validate an authentication token before allowing sensitive operations. Administrative operations such as creating doctor accounts are restricted to the administrator role, while doctor and patient functions are exposed only to their respective authorized users.

9. User Interface

The application uses a clean dashboard-oriented interface. The admin control center provides sidebar navigation and a structured doctor creation form. Patients receive a doctor browsing and appointment booking experience. Doctors receive a dedicated workspace for appointment requests, history, availability, profile information, and patient management.

The screenshot shows the 'Add Doctor' form in the Admin Control Center. The form is structured as follows:

- ACCOUNT CREDENTIALS**
 - Doctor Name *: smith
 - Email Address *: smith@doc.com
 - Phone Number *: 9876551222
 - Temporary Password *: *****
- MEDICAL PRACTICE DETAILS**
 - Qualification *: e.g. MBBS, MD (Cardiology)
 - Specialization *: e.g. Cardiologist
 - Experience (Years) *: e.g. 10
 - Hospital / Clinic *: e.g. City General Hospital
 - Consultation Fee (\$) *: e.g. 100
 - Available Days *: e.g. Mon, Wed, Fri
 - Available Slots *: (empty)

Figure: Admin Control Center – Add Doctor

The screenshot shows the 'Add Doctor' form in the Admin Control Center, with the form filled out. The form is structured as follows:

- ACCOUNT CREDENTIALS**
 - Phone Number *: 9876551222
 - Temporary Password *: *****
- MEDICAL PRACTICE DETAILS**
 - Qualification *: MBBS
 - Specialization *: ophthalmologist
 - Experience (Years) *: 2
 - Hospital / Clinic *: general hospital
 - Consultation Fee (\$) *: 1000
 - Available Days *: mon, fri
 - Available Slots *: 9:00

A 'Create Doctor Account' button is visible at the bottom right of the form.

Figure: Doctor Account Details

The screenshot shows a web browser window with the URL 'book-a-doctor-beta.vercel.app/register'. The page features a central white card titled 'Create Patient Account' with the subtitle 'Join us to book appointments with certified medical doctors.' The form includes the following fields: 'Full Name *' (text input with 'satya'), 'Email Address *' (text input with 'satya@gmail.com'), 'Phone Number *' (text input with '98765569'), 'Password *' and 'Confirm Password *' (password inputs with masked characters), and 'Gender *' (a dropdown menu showing 'Select Gender'). A blue 'Sign Up →' button is at the bottom of the form, and a link 'Already have an account? Sign in' is located below the button.

Figure: Patient Registration

The screenshot displays the 'Book an Appointment' page in the 'BookaDoctor' application. The browser window shows the URL 'book-a-doctor-beta.vercel.app/patient/book'. On the left is a sidebar with navigation links: 'Dashboard', 'Book Appointment' (highlighted), 'My Appointments', 'Medical Reports', 'Notifications', 'Profile', and 'Settings'. At the bottom of the sidebar is a user profile for 'satya' with the email 'satya@gmail.com' and a 'Logout' button. The main content area is titled 'BOOK APPOINTMENT' and 'Book an Appointment', with the subtitle 'Search and schedule clinical consultation slots with verified medical practitioners.' It features a search bar with the placeholder 'Search by doctor name, specialty, or clinic...'. Below the search bar are four doctor cards, each with a profile picture, name, specialty, 0.0 rating, and details on experience, hospital, and fee. Each card has a 'Book Appointment' button. The doctors listed are: Dr. Alice Chambers (Cardiologist, 10 Years Experience, city hospital, \$500), Dr. navya (Cardiologist, 10 Years Experience, city hospital, \$100), Dr. mallidi satya (Ophthalmologist, 10 Years Experience, city hospital, \$100), and Dr. smith (Ophthalmologist, 2 Years Experience, general hospital, \$1000). The top right of the page shows the date 'Tuesday, Jul 14, 2026' and the user's 'Health Portal: satya'.

Figure: Book an Appointment

10. Testing

The project can be validated using functional, API, integration, and user-interface testing. Major workflows include login validation, role access, doctor account creation, patient registration, doctor retrieval, appointment submission, appointment status handling, and patient information display.

Test Area	Example Validation
Authentication	Valid/invalid credentials and role access
Doctor Management	Mandatory fields and doctor creation
Patient Registration	Input validation and account creation
Appointment Booking	Doctor selection and appointment submission
Doctor Dashboard	Request visibility and patient details
API	Status codes, request payloads, and responses

11. Screenshots or Demo

The following screenshots were captured from the submitted project demonstration recording and show the main role-based workflows of the Book A Doctor application.

The screenshot displays the 'Book Appointment' interface for a patient. On the left is a sidebar with navigation links: 'Book Appointment' (active), 'My Appointments', 'Medical Reports', 'Notifications', 'Profile', and 'Settings'. Below the sidebar is a user profile for 'satya' with a 'Logout' button. The main content area shows the appointment details for 'Dr. Smith, OPTHAMOLOGIST' at 'general hospital'. The form consists of several sections: '1. SELECT CONSULT DATE *' with a dropdown showing 'Friday, Jul 17, 2026'; '2. SELECT TIME SLOT *' with a button for '9:00 am to 4:00 pm'; 'Reason for Consultation *' with a dropdown for 'regular checkup'; 'SYMPTOMS REPORTED (OPTIONAL)' with a text area; 'ADDITIONAL NOTES (OPTIONAL)' with another text area; and a payment section labeled '\$ Due at Hospital Registration:' with a value of '\$1000'. At the bottom right are 'Cancel' and 'Confirm Appointment' buttons.

Figure: Appointment Details

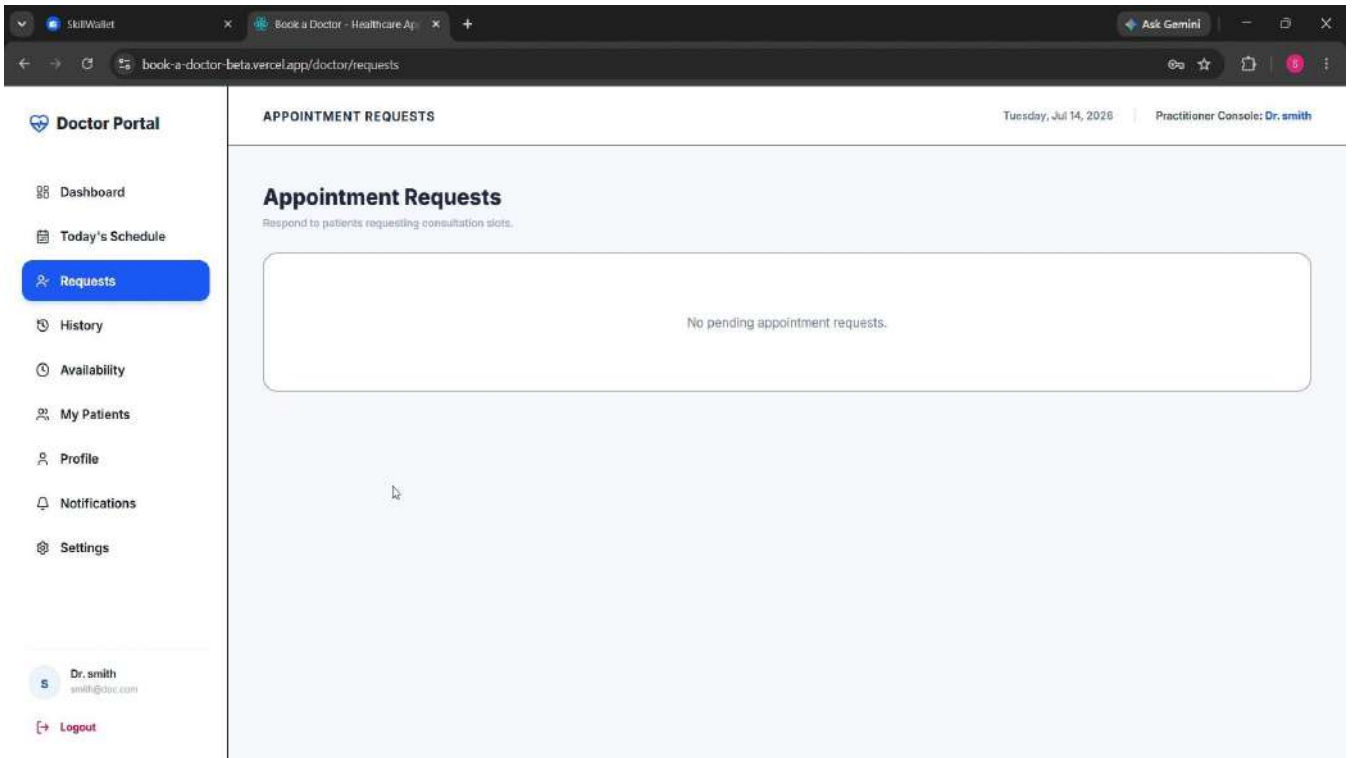


Figure: Doctor Appointment Requests

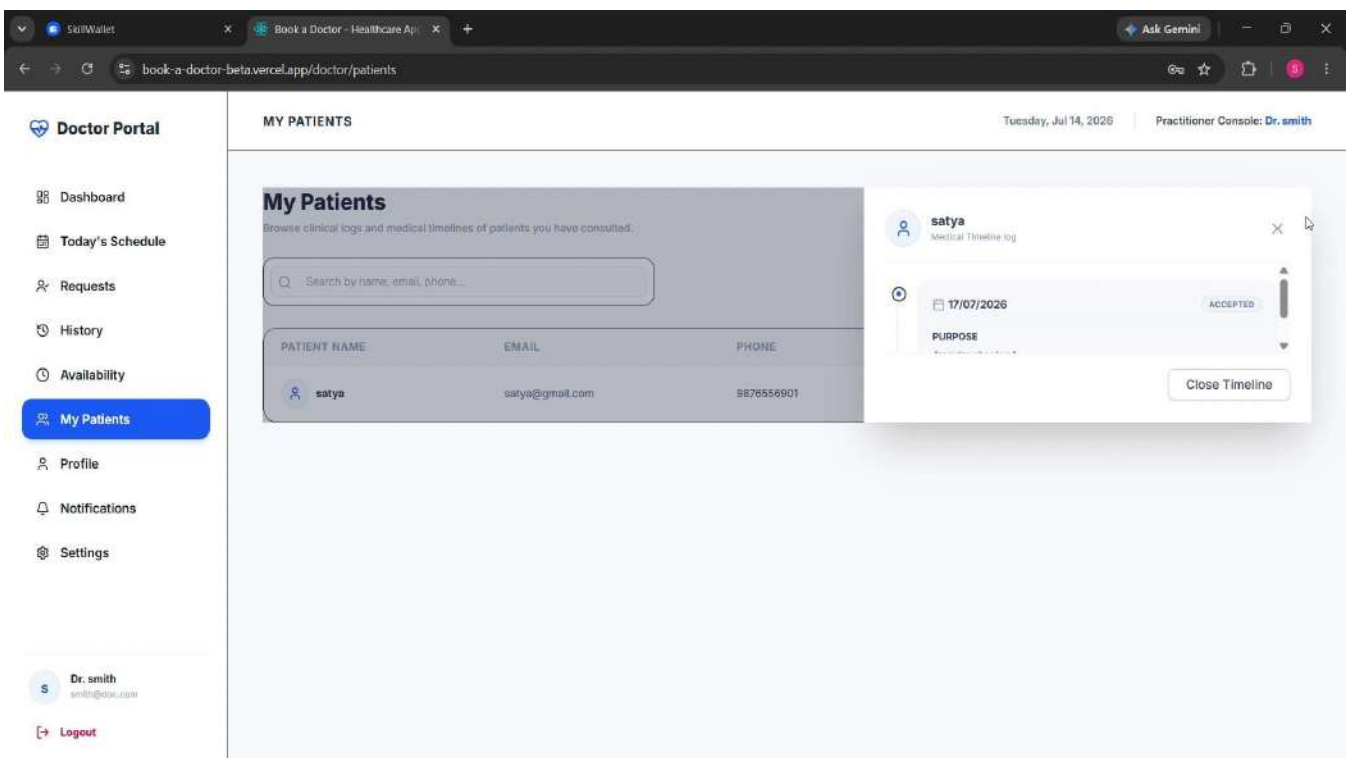


Figure: Doctor – My Patients

12. Known Issues

- Some settings or secondary modules may currently be under development.
- Appointment availability and real-time schedule conflict handling can be improved.
- Form validation messages and edge-case handling may require additional refinement.
- Exact API endpoint documentation should be synchronized with the final source code before submission.

13. Future Enhancements

- Add online video consultation support.
- Integrate email, SMS, or push appointment notifications.
- Add online payment and consultation fee processing.
- Implement doctor ratings and patient feedback.
- Provide advanced doctor search by location, specialty, fee, and availability.
- Add prescription and medical report management.
- Implement real-time calendar scheduling and conflict prevention.
- Deploy the application to a cloud platform with production monitoring and security controls.

Conclusion: Book A Doctor demonstrates a practical MERN full stack application with multiple user roles and an end-to-end healthcare appointment workflow. The project combines frontend interfaces, backend services, data storage, authentication, and role-specific features in a single integrated system.